

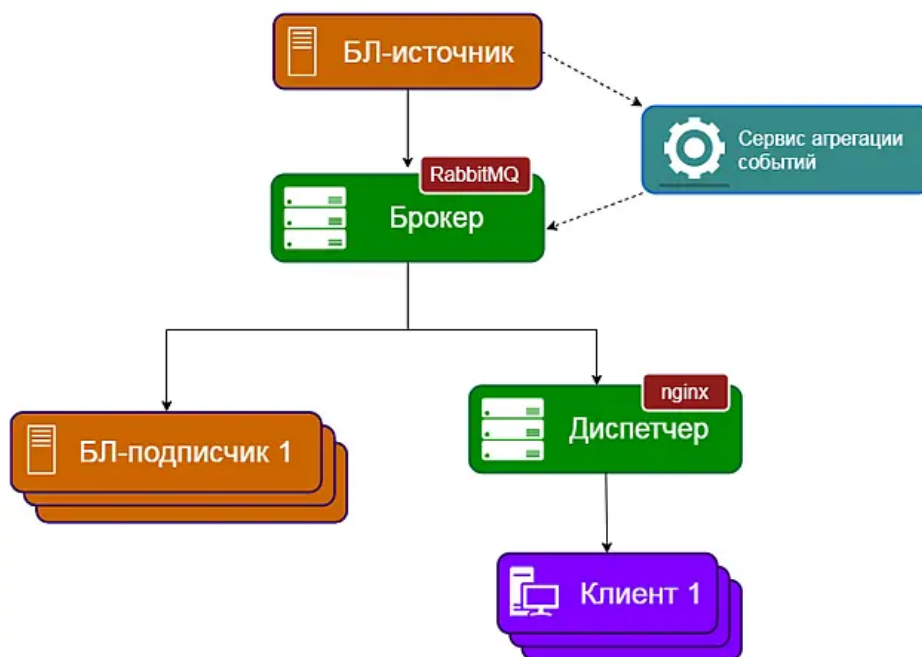
Работа с сервисными событиями в облаке

В Wasaby Framework существует механизм, который позволяет при наступлении событий в сервисах бизнес-логики (далее по тексту — БЛ) оповещать об этом клиентские приложения и другие сервисы БЛ.

Преимущества данного механизма:

1. Повышение скорости реакции приложения на изменение данных.
2. Снижение нагрузки на сервис БЛ за счет адресной рассылки информации об изменениях только тем, кто в ней заинтересован, и только тогда, когда она доступна.
3. Ослабление логических зависимостей между узлами в составе облака.

Общая схема механизма оповещения представлена на следующем изображении:



Комментарий к схеме:

1. БЛ-источник.

Сервис БЛ, на котором [публикуют события](#) и оповещают подписчиков о наступлении событий.

2. Брокер.

Осуществляет обмен сообщениями между компонентами программной системы. В нашем случае брокером является [RabbitMQ](#).

3. БЛ-подписчик.

Другой сервис или группа серверов, заинтересованная в событиях БЛ-источника.

4. Диспетчер.

Балансировщик нагрузки на сервер, перенаправляет сообщения событий.

5. Клиент.

Клиентское приложение (вкладка веб-браузера, СБИС Плагин), заинтересованное в событиях БЛ-источника.

6. Сервис агрегации событий.

Позволяет избежать избыточности событий и множественных обращений, объединяя несколько событий в одно. Подробнее см. [здесь](#).

Настройка облака для работы с событиями

1. Подключите [сервисные модули](#):

- EventSubscription в сервисы, в которых планируется использование механизма оповещения о событиях.
- EventSubscription-Py в сервисы, в которых планируется подписка на события из исходного кода на Python.
- В сервисы должны быть подключены и другие сервисные модули, от которых зависит работа перечисленных выше. Список зависимостей сервисного модуля указан в его свойствах (Properties) в опции миксинов Module dependencies.

2. Настройка адресации серверов брокеров задаётся через настройку кластеров Облака.

Правила именования события

События должны именоваться по правилу `<service>.<eventoccured>[:<ident>]`, где

- `<service>` — название сервиса, из которого публикуется событие. Запрещено использование точки в названии сервиса в качестве разделителя.
- `<eventoccured>` — обозначение действия. Выберите такое название, которое точно характеризует произошедшее событие.
- `<ident>` — необязательный идентификатор, который может быть использован в прикладном коде. Например, его применяют для отделения событий по разным ресурсам или номерам задач.

Для именования используйте латинские буквы нижнего регистра, цифры и точку.

Точку рекомендуется использовать для выделения "областей", например, когда у нас есть много событий с одним префиксом: `shaitan.clienthasbeencreated`, `shaitan.clienthasbeendeleted` или `shaitan.clienthasbeenedited`.

В этом случае лучше сделать так: `shaitan.client.created`, `shaitan.client.deleted` или `shaitan.client.edited`.

В `<service>` и `<eventoccured>` должны начинаться на букву или цифру.

В `<ident>` допускается использование символа доллар.

Примеры: `shaitan.client.generated`, `reclamation.taskhasbeenadded:35`, `myservice.client.successfullycreated`.

Сценарии доставки события

В приведенной выше схеме выделяют два сценария работы, которые различаются по типу подписчика.

Сервисные события

Получатели: сервисы.

Участники: БЛ-источник, Брокер и БЛ-подписчик.

БЛ-подписчиком, как правило, выступает группа серверов. Если подписчик — это единичный экземпляр сервиса БЛ, то используется [дополнительный синтаксис](#).

В общем случае на сервисные события нельзя подписываться на стороне бизнес-логики основного сервиса.

Межсервисные события обезличены и при такой подписке обработчик будет вызван на каждом юните основного сервиса под служебным пользователем, который не привязан ни к какой схеме.

Если нужно обработать событие в рамках какого-то определенного клиента, то вместо публикации события необходимо совершить асинхронный вызов в основной сервис под этим клиентом.

Обработка события в рамках каждого клиента не предусмотрена. Однако, можно вынести этот функционал в отдельный сервис и подписываться там.

Если нужно обработать событие в рамках ограниченного набора клиентов, который в момент публикации неизвестен, то необходимо воспользоваться [сервисом массовых рассылок](#).

При генерации запроса на добавление подписки вся группа серверов БЛ-подписчика выступает как одно целое (если явно не указано обратное), т.е. поступающие из канала сообщения могут быть доставлены любому экземпляру в составе группы. При создании подписки БЛ-подписчик может установить режим доставки сообщений. При работе в режиме с гарантированной доставкой не полученные группой серверов сообщения хранятся на Брокере. Остановка или перезапуск любого количества серверов в составе группы не ведет к потере подписки.

Оповещение о наступлении события в данном сценарии со стороны БЛ-источника производится аналогично сценарию "Подписчик "Клиент" с той лишь разницей, что "клиентами" здесь по умолчанию являются целые группы серверов.

Если в момент отправки сообщения группа (экземпляр) недоступна, и выбран режим без гарантированной доставки, то сообщение ей доставлено не будет.

В стандартном случае публикация события происходит в одном рабочем процессе (`sbis-grpc-worker300.exe`), а его обработка — в другом. Но обмен сообщениями события осуществляется через Брокер. При этом допускается, что источником и подписчиком на событие может выступать один и тот же сервис БЛ.

Существует схема, когда событие публикуется и обрабатывается [внутри одного рабочего процесса](#). В этом случае брокер уже не участвует в обмене сообщениями события.

Гарантии доставки сервисных событий

В [схеме доставки](#) сервисных событий условно можно выделить два сценария:

1. События не агрегируются: Источник события → Брокер → Подписчик.
2. События агрегируются: Источник события → Сервис агрегации → Брокер → Подписчик.

Брокер и Сервис агрегации по умолчанию сохраняют данные о событиях в оперативной памяти сервера, на котором они запущены. Напомним, что оперативная память является энергозависимой, поэтому при аппаратном или программном сбое сервера она будет очищена, а данные о событиях — потеряны. В итоге, клиент не получит сообщения о событии.

Брокер поддерживает гарантированную доставку событий. Для этого необходимо, чтобы при подписке на событие был установлен флаг либо атрибутом [durable](#) (в Python), либо методом [SetIsDurable](#) (в C++). В этом случае событие, полученное брокером, будет сохраняться в БД (энергонезависимое хранилище) и, как следствие, гарантированно доставлено до подписчика даже после перезапуска сервиса.

Для [Сервиса агрегации событий](#) не существует понятия гарантированной доставки, потому что он не осуществляет доставку событий до подписчика.

Пример публикации сервисного события

[API для публикации событий](#)

- C++

```
1 #include <sbis-event-subscription/publish.hpp>
2 #include <sbis-event-subscription/event.hpp>
3 ...
4 event::Event e( L"some.event", event::evSERVER_ONLY );
5 e.SetPayload( message );
6 e.SetPolicy( PublicationPolicy::ppIMMEDIATELY );
7 event::Publish( e );
```

- Python

```
1 import event
2 ...
3 event.Publish(event.Event(
4     name="some.event",
5     payload=message,
6     visibility=event.Visibility.evSERVER_ONLY,
7     policy=sbis.PublicationPolicy.ppIMMEDIATELY) )
```

Клиентские события

Получатели: клиенты

Участники: БЛ-источник, Брокер, Диспетчер и Клиент.

Для [подписки на событие](#) клиентское приложение должно сделать запрос к диспетчеру ([nginx](#)), в котором указать канал, в который будут публиковаться сообщения о событии.

Взаимодействие клиентского приложения и канала происходит по протоколу [STOMP](#), работающему поверх одного из протоколов: [SockJS](#) или [WebSocket](#). Выбор этих протоколов выполняется автоматически и зависит от клиента и его версии.

При наступлении события Брокер оповещает Диспетчер, который далее оповещает клиентские приложения. При разрыве соединения на стороне Клиента (например, обновление страницы) подписка теряется и должна быть установлена снова.

Клиентские события на сервисе публикуются асинхронно относительно потока управления. Возврат из функции Publish не означает, что сообщение опубликовано. Это означает, что сообщение принято и будет опубликовано. В случае проблем с брокером сообщение может быть опубликовано позже или не будет публиковаться вообще, т.е. если STOMP-брокер не работает, методы публикации не зависят и не генерируют ошибки в адрес вызвавшего.

Буфер под накопление событий ограничен для публикации клиентских событий в каждом рабочем процессе сервиса БЛ. Если система не успевает публиковать события, то начинают отбрасываться самые старые события из неопубликованных.

Соответственно, если необходимо дождаться публикации события, вы можете использовать функцию [WaitForPublish](#). Это означает, что вы можете подождать в течение заданного времени



Важно помнить, что ожидание публикации события не гарантирует, что событие получено клиентом, это означает, что событие получено брокером.

Если набирается больше 1000 неисполненных публикаций от одного RPC-вызова, то в течение периода не более секунды основной поток управления ждет окончания их публикации. Это мера борьбы с накоплением "долгов" в рабочих процессах.

Политика доставки сообщений до Клиента



Клиент не должен отсутствовать более 5 минут

Доставка сообщений до Клиента гарантируется только в случае его активного подключения к сервису STOMP с учётом небольшого интервала переподключения. Для веб-браузера активное подключение означает, что вкладка с веб-приложением открыта. Если Клиент отсутствует более 5 минут (например, вкладка веб-браузера закрыта), то все накопившиеся для него данные будут удалены из памяти (очередь подключения будет удалена).



Очередь подключения не превышает 300 сообщений

Для предотвращения неограниченного роста потребляемой памяти очередь ограничена 300 сообщениями. Более старые сообщения будут вытесняться более новыми при превышении этого ограничения.



Ограничение политики

Данная политика не распространяется на случай, когда подписчиком является другой сервис или группа сервисов.

Гарантии и особенности доставки сервисных событий подробно описаны в статье [Доставка сервисных событий до клиента](#).

Уровень изоляции события

На уровне брокера предусмотрена система фильтрации событий, доставляемых клиентским приложениям, благодаря которой клиентское приложение получает сообщения о событиях в соответствии с уровнем изоляции, заданным при [публикации события](#). Существует три уровня изоляции:

- **Global** - события данного уровня изоляции получают все клиентские приложения. Область применения ограничена. Такие события могут порождать большую нагрузку и использование данного уровня изоляции должно быть согласовано.
- **Client** - события данного уровня изоляции получают все клиентские приложения определенного [клиента](#) в облаке. Область применения: отправка уведомления об обновлении клиента в процессе обновления облака, уведомление о новостях и прочее.
- **User** - события данного уровня изоляции получают все клиентские приложения определенного [пользователя](#) в облаке. При публикации события устанавливается список пользователей, для которых будет доставлено событие. Область применения: сообщения в чате, нотификация о звонке, поощрения и взыскания, изменение котировок торгов и прочее. Данный уровень изоляции должен быть самым часто используемым, потому что нагрузка на сервер в виде трафика значительно снизится по сравнению с той схемой, когда периодически вызывается метод, проверяющий изменения на сервере.

Если в приложении на получение события подписался пользователь X клиента Y, то событие будет ему доставлено, если справедливо одно из следующих утверждений:

- при публикации был выбран уровень изоляции **Global**;
- при публикации был выбран уровень изоляции **Client**, клиент Y указан в списке получателей события;
- при публикации был выбран уровень изоляции **User**, пользователь X указан в списке получателей события.



Использование глобальных событий требует согласования

Частая публикация события большому числу пользователей приведёт к большой нагрузке на сервис STOMP (будет большой коэффициент усиления нагрузки, что не может ни сказаться на стабильности системы).

Использование глобальных событий создаёт существенную и непредсказуемую нагрузку на кластер клиентских событий, поэтому любое их применение нужно согласовать с [Головановым К.А.](#) и [Смирновым И.В.](#).

Согласование требуется в следующих случаях:

1. если вы хотите использовать событие впервые;
2. если ранее был согласован алгоритм применения события, и вы его хотите изменить.

Типы публикуемых событий

Кроме уровня изоляции выделяется еще такое понятие как тип события. Различают **события общего типа** и **события в адрес приложения**.

События общего типа используются наиболее часто. Для их доставки используется единая подписка (общий канал) для пользователя, что позволяет оптимизировать механизм подписки (не требуется отправка на сервер сообщения о необходимости подписаться для каждого имени события в отдельности) и упростить маршрутизацию события на сервере. Общий канал создается при инициализации первой подписки со стороны клиента.

Обратной стороной такого подхода является тот факт, что клиент, подписавшийся на общий канал, получает все события, опубликованные в сторону пользователя и доставленные согласно уровню изоляции. Фильтрация таких событий по имени происходит на клиенте, а не на сервере. Большинство доставляемых событий в таком случае не нужно клиенту в конкретный момент времени.

Несмотря на простоту использования, такой тип расходует большое количество трафика, также на мобильных устройствах быстро садится заряд батареи питания. В связи с этим, был установлен запрет на использование событий общего типа при взаимодействии с мобильными приложениями, об этом, в частности, говорилось в [новости](#).

В отличие от событий общего типа, при использовании **событий в адрес приложения** для каждого события явно создаётся и удаляется подписка (отправляется соответствующее сообщение на сервер STOMP). Данный тип событий доставляется только при подписке на него.

В настоящий момент единственным возможным значением приложения является `mobile`. При подписке также должно указываться это имя приложения (`mobile`). **Это название не нужно трактовать как жесткое требование.**

Использование API `SetApplication` меняет политику доставки события до потребителя с "общего канала" на "только по списку подписавшихся". В офлайн-приложениях поддерживаются **только события в адрес приложения**, а для `web` - оба типа.

Примеры публикации события

Пример публикации события по идентификатору персоны из Сервиса профилей

Идентификатор персоны существует в рамках [Сервиса профилей](#). Тип значения идентификатора — [UUID](#).



При работе с веб-приложением online.sbis.ru вы могли видеть такой идентификатор, когда заходили на персональную страницу пользователя. С главной страницы приложения перейдите в меню "Сотрудники", затем в списке найдите любую запись о персоне и кликните по ней. Будет открыта панель с данными о персоне. В верхнем меню выберите пункт "Ссылка" → "Получить ссылку".

В результате в буфер обмена будет сохранена ссылка вида <https://online.sbis.ru/person/9943efc7-7e01-473f-b80f-354fe7247359>, где последняя часть после символа "косая черта" (/) — "9943efc7-7e01-473f-b80f-354fe7247359" — является идентификатором персоны.

Событие будет опубликовано для всех пользователей, которые привязаны к персоне или персонам.

- C++

```
1 #include <sbis-event-subscription/publish.hpp>
2 #include <sbis-event-subscription/event.hpp>
3 // Идентификатор персоны.
4 String person_id;
5
6 event::Event e( L"some.event", event::evCLIENT_ONLY );
7 e.SetPayload( message );
8 e.SetPolicy( PublicationPolicy::ppIMMEDIATELY );
9 e.SetPerson( person_id );
10 event::Publish( e )
11
12 // Массив идентификаторов персон.
```

```

13 | ArrayString persons;
14 |
15 | event::Event e( L"some.event", event::evCLIENT_ONLY );
16 | e.SetPayload( message );
17 | e.SetPolicy( PublicationPolicy::ppIMMEDIATELY );
18 | e.SetPersons( persons );
19 | event::Publish( e )

```

- Python

```

1 | import event
2 |
3 | # Идентификатор персоны.
4 | person_id = "9943efc7-7e01-473f-b80f-354fe7247359"
5 |
6 | event.Publish(event.Event(
7 |     name="some.event",
8 |     payload=message,
9 |     visibility=event.Visibility.evCLIENT_ONLY,
10 |    policy=sbis.PublicationPolicy.ppIMMEDIATELY,
11 |    persons=[person_id]))

```

```

1 | import event
2 |
3 | # Идентификаторы персон.
4 | person_ids = ["9943efc7-7e01-473f-b80f-354fe7247359"]
5 |
6 | event.Publish(event.Event(
7 |     name="some.event",
8 |     payload=message,
9 |     visibility=event.Visibility.evCLIENT_ONLY,
10 |    policy=sbis.PublicationPolicy.ppIMMEDIATELY,
11 |    persons=person_ids))

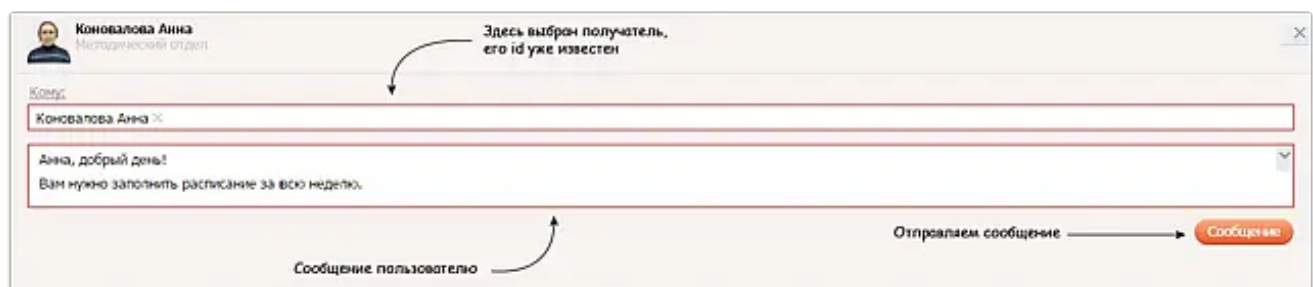
```

Пример публикации в адрес клиента

[API для публикации событий](#)

Рассмотрим прикладной пример, описывающий публикацию события в облаке. Он относится к ситуации, когда один пользователь отправляет сообщение другому пользователю, например как на online.sbis.ru.

Диалог отправки сообщения выглядит следующим образом:



Сначала в качестве адресата пользователь выбирает из предложенного списка получателя сообщения. Контроль, который предоставляет список пользователей в системе, при выборе получателя запоминает его идентификатор из таблицы БД. Идентификатор потом будет передан в функцию отправки сообщения (далее по тексту — `sendMessage`) переменной `user_id`.

Далее пользователь вводит текст сообщения, который будет передан в функцию `sendMessage` переменной `user_message`, и нажимает кнопку "Сообщение". Клик по кнопке, в свою очередь, на бизнес-логике вызывает `sendMessage`:

- C++

```

1 | # include <sbis-event-subscription/publish.hpp>
2 | # include <sbis-event-subscription/event.hpp>
3 | ...

```

```

4 void sendMessage (... , identificator, message)
5 {
6     ...
7     // Публикуем событие об отправке нового сообщения.
8     event::Event ev( L"edo.messagehasbeensent", event::evCLIENT_ONLY
9 );
10    ev.SetPayload( message );
11    ev.SetPolicy( PublicationPolicy::ppIMMEDIATELY );
12    ev.SetUser( identificator );
13    event::Publish( ev );
14    ...
15 }
16 sendMessage(... , user_id, user_message);

```

- Python

```

1 import event
2 ...
3 def sendMessage (self, ... , identificator, message):
4     ...
5     # Публикуем событие об отправке нового сообщения.
6     event.Publish( event.Event(name="edo.messagehasbeensent",
7         payload=message,
8         visibility=event.Visibility.evCLIENT_ONLY,
9         policy=sbis.PublicationPolicy.ppIMMEDIATELY,
10        users=[identificator]
11    ) )
12    ...
13 )
14 ...
15 sendMessage(... , user_id, user_message)

```

Пример публикации события в адрес одного приложения

Чтобы опубликовать **событие для одного** мобильного приложения, нужно использовать метод [SetApplication\(\)](#) (для C++) или атрибут [application](#) (для Python), где явно установить ограничение — псевдоним такого приложения-получателя (на данный момент только mobile).

- C++

```

1 #include <sbis-event-subscription/publish.hpp>
2 #include <sbis-event-subscription/event.hpp>
3 event::Event e( L"contacts.communicator", event::evCLIENT_ONLY );
4 e.SetPayload( payload );
5 e.SetPolicy( PublicationPolicy::ppIMMEDIATELY );
6 e.SetUsers( users );
7 e.SetApplication( L"mobile" );
8 event::Publish( e )

```

- Python

```

1 import event
2 event.Publish(event.Event(
3     name="contacts.communicator",
4     payload=payload,
5     visibility=event.Visibility.evCLIENT_ONLY,
6     policy=sbis.PublicationPolicy.ppIMMEDIATELY,
7     users=users,
8     application="mobile"))

```

Подход адресной публикации событий ускоряет доставку оповещения получателям и экономит ресурсы сервера. Такие события не будут приходить в интерфейс или в другие приложения, например событие опубликованное для мобильного приложения не придет в СБИС Плагин.



При подписке со стороны мобильных и desktop-приложений вызов метода `SetApplication()` не выполняется. При публикации, если необходимо доставить сообщение в какое-либо offline-приложение, то нужно обязательно указать атрибут `application=mobile`.

API

Массовая публикация клиентских событий



Важно!

Для публикации события нескольким получателям рекомендуется также использовать класс `event::Event` (в Python - `event.Event`). В этом случае для указания получателей в C++ нужно использовать вместо методов `SetUser`, `SetClient` и `SetPerson` соответствующий метод `SetUsers`, `SetClients`, `SetPersons`. В Python требуется просто передать несколько получателей в соответствующий аргумент конструктора класса `event.Event` (`users`, `clients` или `persons`). В целях обратной совместимости сохранена также функция `MulticastPublish`. Её использование в новом коде не рекомендуется.

Ниже представлены примеры вызова функции `MulticastPublish`. Подробнее читать [здесь](#).

• C++

```
1 #include <sbis-event-subscription/publish.hpp>
2 ...
3 void NewsPublication (... , identifiers, message)
4 {
5     ...
6     // Публикуем событие о новости.
7     event::MulticastPublish(
8         L"inside.newshasbeenpublished",
9         message,
10        event::evCLIENT_ONLY,
11        PublicationPolicy::ppIMMEDIATELY,
12        event::evUSER,
13        identifiers
14    );
15    ...
16 }
17 ...
18 NewsPublication(... , user_id_array, text_news);
```

• Python

```
1 import event
2 ...
3 def NewsPublication (self, ... , identifiers, message):
4     ...
5     # Публикуем событие о новости.
6     event.MulticastPublish(
7         "inside.newshasbeenpublished",
8         message,
9         event.Visibility.evCLIENT_ONLY,
10        sbis.PublicationPolicy.ppIMMEDIATELY,
11        event.Delivery.evUSER,
12        identifiers
```

```

13     )
14     ...
15 )
16 ...
17 NewsPublication(... , user_id_array, text_news)

```

Ожидание публикации клиентского события

Для ожидания публикации события используется функция `WaitForPublishing`. Она позволяет дождаться публикации в брокер STOMP всех ранее заказанных на публикацию клиентских событий в рамках текущего RPC-вызова в течение указанного таймаута.

- [C++](#)

```

1  /**
2  * @param timeout - максимальное время ожидания
3  * @return true если все события были отправлены, false если истекло
   время ожидания
4  */
5  SBIS_EVENT_SUBSCRIPTION300_PUBLIC bool WaitForPublishing( const
   DateTimeDelta& timeout );

```

- [Python](#)

```

1  def WaitForPublishing(timeout):
2      '''
3      :param timeout: Таймаут ожидания.
4      :type timeout: datetime.timedelta
5      :return: True если все события были отправлены, False если истекло
   время ожидания.
6      :rtype: bool
7      '''

```

Событийные операции с контрактом и fallback

Вы можете организовать вызов метода БЛ из клиентской части веб-приложения таким образом, чтобы на получение ответа был установлен таймаут, при превышении которого выполняется аварийный сценарий обработки результата.

Вызванный метод БЛ передаётся в сервис БЛ на выполнение, а веб-браузеру возвращается HTTP-статус 200 ОК. Вследствие освобождается занятый веб-сокет, который может быть использован для вызова других методов БЛ, что может увеличить быстродействие приложения. Если метод успевает выполниться в заданный таймаут, то срабатывает `callback`, если же нет — `errback`.

Сценарии вызова с fallback'ом рекомендуется выполнять для длительных методов БЛ. Например, сценарий, когда пользователи загружают свои файлы на сервис БЛ для их разбора и анализа.



В чём недостаток обычного вызова метода БЛ?

Обычные вызовы методов БЛ дожидаются ответа сервера, занимая по одному сетевому соединению на запрос. Недостаток такого подхода заключается в том, что, как правило, браузеры ограничивают количество одновременно задействованных веб-сокетов для взаимодействия с определенным доменом (например, *.sbis.ru). Это означает, что пока мы ожидаем ответа сервера на длительный запрос, браузер будет подгружать данные с нашего сайта при помощи меньшего количества сетевых соединений. Это может существенно замедлить работу веб-страницы.



Ограничение использования

Описанный способ вызова метода БЛ добавляет существенные накладные расходы на передачу ответа от сервера клиенту, а потому должен использоваться лишь в том случае, когда это действительно необходимо и позволит улучшить опыт взаимодействия пользователя с вашим приложением. Перед применением механизма требуется проконсультироваться с ответственным разработчиком — [Бойцов Е.А.](#).

Для реализации прикладных сценариев с fallback'ом следует использовать класс [Transport/RPCJSON](#).

```
1 require(['Transport/RPCJSON'], function(rpcjson) {
2     var rpcjson = new RPCJSON({
3
4         // Работа в режиме обработки таймайту и запуска аварийного
сценария.
5         fallback: true,
6
7         // Таймаут, в мс.
8         timeout: 120000
9     });
10    res = rpcjson.callMethod('Объект.Метод', params);
11    res.addCallbacks(
12
13        // Обработка ответа.
14        function(res) { ...},
15
16        // Аварийный сценарий обработки результата – таймаут получения
ответа.
17        function(res) { ...}
18    );
19 });
```